

# 03 — TOAST (The Oversized-Attribute Storage Technique)

**Module:** PostgreSQL Internals | **Difficulty:** Advanced | **Est. reading time:** 45 min

---

## Table of Contents

1. [Learning Objectives](#)
  2. [Overview](#)
  3. [The TOAST Threshold](#)
  4. [TOAST Strategies](#)
  5. [Compression Algorithms: pglz vs lz4](#)
  6. [Out-of-Line Storage: pg\\_toast \\* Tables](#)
  7. [TOAST Table Schema: chunk\\_id / chunk\\_seq / chunk\\_data](#)
  8. [How Queries Transparently De-toast](#)
  9. [Varlena Header Encoding](#)
  10. [ASCII Diagrams](#)
  11. [Live Observation Queries](#)
  12. [When TOAST Hurts Performance](#)
  13. [Common Mistakes](#)
  14. [Best Practices](#)
  15. [Performance Considerations](#)
  16. [Interview Questions](#)
  17. [Exercises with Solutions](#)
  18. [Cross-References](#)
- 

## Learning Objectives

After completing this section you will be able to:

- Explain the TOAST threshold and the order in which PostgreSQL tries strategies.
  - Distinguish between PLAIN, EXTENDED, EXTERNAL, and MAIN storage strategies and state when to use each.
  - Describe the physical structure of a TOAST table and the three columns every one has.
  - Trace the path of a large value from an INSERT statement to its final storage location.
  - Explain why TOAST is transparent to SQL queries and where the de-toasting overhead occurs.
  - Identify slow queries caused by TOAST access and propose mitigation strategies.
- 

## Overview

PostgreSQL's heap pages are fixed at 8 KB. A single tuple must fit within one page (approximately). Variable-length columns — `text`, `varchar`, `bytea`, `json`, `jsonb`, `xml`, arrays, and others — can easily exceed this limit.

TOAST (The **O**versized-**A**tttribute **S**torage **T**echnique) is PostgreSQL's transparent solution: large values are automatically compressed and/or stored outside the main heap in a companion "TOAST table". From SQL's perspective, the data is just there — no special syntax is needed to read or write large values.

The technique was introduced in PostgreSQL 7.1 and has been the foundation of unlimited row width ever since.

## The TOAST Threshold

PostgreSQL will attempt to TOAST a value when the tuple's total size (all columns combined) would exceed the **TOAST threshold**, which is:

```
TOAST_TUPLE_THRESHOLD = BLCKSZ / 4 = 8192 / 4 = 2048 bytes
```

So approximately **2 KB**. More precisely, the threshold is `MAXALIGN(TOAST_TUPLE_THRESHOLD)` .

This does not mean individual values over 2 KB get toasted — it means the *whole tuple* must fit in 2 KB before PostgreSQL decides some values need to be pushed out. PostgreSQL will try to process large columns (preferring those with `EXTENDED` or `MAIN` strategy) until the tuple fits.

There is also a hard maximum: a toasted pointer (18 bytes) must fit in the tuple instead of the value. Once a value is toasted-out-of-line, only an 18-byte struct occupies the heap.

## TOAST Strategies

Each column has a storage strategy stored in `pg_attribute.attstorage` :

| Strategy | attstorage | Compress?   | Out-of-line?      | Description                                                                                            |
|----------|------------|-------------|-------------------|--------------------------------------------------------------------------------------------------------|
| PLAIN    | p          | No          | No                | For small, fixed-width types (int, bool). Never TOASTed.                                               |
| EXTENDED | x          | Yes (first) | Yes (if needed)   | Default for variable-length types. Try compression first; if still too large, store out-of-line.       |
| EXTERNAL | e          | No          | Yes (if needed)   | Store out-of-line without compression. Good for already-compressed data (JPEG, ZIP).                   |
| MAIN     | m          | Yes (first) | Yes (last resort) | Prefer to keep in main tuple (compress first). Only push out-of-line if the page truly cannot hold it. |

### Processing order when a tuple is too large:

1. Compress EXTENDED and MAIN columns (pglz or lz4).
2. Move EXTENDED columns out-of-line (largest first).
3. Move MAIN columns out-of-line (last resort).
4. PLAIN and EXTERNAL already-in-line columns stay in the heap.

```
-- Change storage strategy for a column
ALTER TABLE documents ALTER COLUMN body SET STORAGE EXTERNAL;
-- (Does not rewrite existing rows – takes effect for new/updated rows)

-- Check current strategies
```

```

SELECT attname, attstorage,
       CASE attstorage
         WHEN 'p' THEN 'PLAIN'
         WHEN 'e' THEN 'EXTERNAL'
         WHEN 'm' THEN 'MAIN'
         WHEN 'x' THEN 'EXTENDED'
       END AS strategy
FROM pg_attribute
WHERE attrelid = 'documents'::regclass
      AND attnum > 0
      AND NOT attisdropped;

```

## Compression Algorithms: pglz vs lz4

### pglz (PostgreSQL LZ)

- The original and still-default algorithm.
- Compression ratio: generally good (comparable to LZ-family algorithms).
- Speed: relatively slow on both compression and decompression paths.
- Available since PostgreSQL 1.x.

### lz4

- Available from PostgreSQL 14+.
- Compression ratio: slightly worse than pglz on average.
- Speed: dramatically faster — decompression is 3–5× faster than pglz, compression 2–3× faster.
- Best choice for frequently accessed large values (JSON documents, logs).

```

-- Set compression algorithm per column (PG 14+)
ALTER TABLE events ALTER COLUMN payload SET COMPRESSION lz4;

-- Or set at the table level for all newly declared columns:
-- default_toast_compression = 'lz4' (in postgresql.conf)

-- Check compression used by stored values
SELECT
  id,
  octet_length(payload)                AS stored_bytes,
  pg_column_compression(payload)       AS algorithm
FROM events
LIMIT 5;

-- Check table-level default
SELECT relname, reltoastrelid
FROM pg_class WHERE relname = 'events';

```

## Out-of-Line Storage: pg\_toast\_\* Tables

When a value must be stored out-of-line, PostgreSQL writes it into a dedicated **TOAST table**. Every table with at least one toastable column gets its own TOAST table, named `pg_toast.pg_toast_<main_table_oid>`.

TOAST tables live in the `pg_toast` schema and are not directly queryable by regular users (though superusers can `SELECT` from them). Each TOAST table also has its own TOAST index for fast chunk lookup by `chunk_id`.

```
pg_toast schema
├─ pg_toast_16385      (TOAST table for main table OID 16385)
│   └─ pg_toast_16385_index  (unique index on chunk_id, chunk_seq)
```

## TOAST Table Schema

Every TOAST table has exactly three columns:

| Column     | Type  | Description                                                                                                                                      |
|------------|-------|--------------------------------------------------------------------------------------------------------------------------------------------------|
| chunk_id   | oid   | The OID of this toasted value. Shared across all chunks of the same value. Acts as the foreign key linking back to the main row's toast pointer. |
| chunk_seq  | int4  | Sequence number of this chunk, starting at 0. Chunks are 2000 bytes each (the <code>TOAST_MAX_CHUNK_SIZE</code> ).                               |
| chunk_data | bytea | The raw bytes of this chunk (possibly compressed).                                                                                               |

A value of 10,000 bytes after compression to 7,000 bytes would be stored as 4 chunks:

- `chunk_seq=0`: 2000 bytes
- `chunk_seq=1`: 2000 bytes
- `chunk_seq=2`: 2000 bytes
- `chunk_seq=3`: 1000 bytes

```
-- Find the TOAST table for a given table
SELECT c.relname AS main_table,
       t.relname AS toast_table,
       t.oid     AS toast_oid,
       pg_size_pretty(pg_total_relation_size(t.oid)) AS toast_size
FROM   pg_class c
JOIN   pg_class t ON t.oid = c.reltoastrelid
WHERE  c.relname = 'documents';

-- Inspect TOAST table contents (superuser only)
SELECT chunk_id, chunk_seq, length(chunk_data) AS chunk_bytes
FROM   pg_toast.pg_toast_16385
ORDER  BY chunk_id, chunk_seq
LIMIT 20;
```

## How Queries Transparently De-toast

When you `SELECT` a column with a toasted value, PostgreSQL:

1. Reads the main heap tuple — the column contains an 18-byte **toast pointer** (a `varattrib_4b` struct with the `VARATT_IS_EXTERNAL` flag set).

2. Checks the toast pointer: is it out-of-line? Compressed in-line? A short inline value?
3. If out-of-line: performs a **sequential scan or index scan on the TOAST table** using the `chunk_id` stored in the pointer, fetching all chunks ordered by `chunk_seq`.
4. Assembles the chunks into the original byte stream.
5. If the value was compressed: decompresses it.
6. Returns the resulting datum to the executor.

This is fully transparent to SQL: `SELECT body FROM documents WHERE id = 1` just works.

## Varlena Header Encoding

Every variable-length value starts with a 1- or 4-byte header that encodes its length and compression/external status:

```
1-byte short form (VARATT_IS_SHORT):
  bits 7-1: length (max 126 bytes of data)
  bit 0: must be 0 (short marker)

4-byte uncompressed:
  bits 31-2: length
  bit 1: 0 (not external)
  bit 0: 1 (this is the "not short" marker)

4-byte compressed (VARATT_IS_COMPRESSED):
  bits 31-2: compressed length
  bit 1: 1 (compressed)
  bit 0: 1

4-byte external pointer (VARATT_IS_EXTERNAL):
  The first byte is 0x01 (external marker)
  Followed by 17 bytes of toast pointer data
```

## ASCII Diagrams

### TOAST Flow

```
INSERT INTO docs (id, body) VALUES (1, '<10 KB HTML string>');
```

Main Heap Page (base/16384/16385)

|                                              |
|----------------------------------------------|
| tuple: id=1   body= [TOAST POINTER 18 bytes] |
| chunk_id=9001                                |

|  
| pointer dereference on SELECT



TOAST Table (pg\_toast/pg\_toast\_16385)

| chunk_id | chunk_seq | chunk_data                 |
|----------|-----------|----------------------------|
| 9001     | 0         | <2000 bytes of lz4 output> |

|      |   |                            |
|------|---|----------------------------|
| 9001 | 1 | <2000 bytes of lz4 output> |
| 9001 | 2 | <2000 bytes of lz4 output> |
| 9001 | 3 | < 543 bytes of lz4 output> |

↓  
lz4 decompress

10,000 bytes of original HTML → returned to client

## In-line Compressed Value

```
INSERT INTO docs (id, body) VALUES (2, '<3 KB text that compresses to 900 bytes>');
```

Main Heap Page

|                                                           |
|-----------------------------------------------------------|
| tuple: id=2   body= [COMPRESSED VARLENA 904 bytes inline] |
| 4-byte header: length=904, compressed=1                   |
| 900 bytes of pglz/lz4 output                              |
| (original 3 KB compressed to 900 B → fits in heap!)       |

## Live Observation Queries

```
-- 1. Create test table and insert large data
CREATE TABLE toast_demo (
  id      serial PRIMARY KEY,
  small   text,
  large   text,
  binary  bytea
);

-- Insert values of different sizes
INSERT INTO toast_demo (small, large, binary) VALUES
('hello',
 repeat('x', 100),          -- 100 bytes: stays inline
 '\x'::bytea),              -- empty
('world',
 repeat('PostgreSQL ', 1000), -- ~11 KB: will be toasted
 repeat('\xDEADBEEF', 500)::bytea); -- ~2 KB binary

-- 2. Check compression
SELECT id,
  pg_column_size(small) AS small_bytes,
  pg_column_size(large) AS large_stored,
  length(large)         AS large_logical,
  pg_column_compression(large) AS compression,
  CASE WHEN pg_column_size(large) < length(large)
    THEN 'compressed'
    ELSE 'not compressed'
```

```

        END AS status
FROM toast_demo;

-- 3. Find TOAST table name
SELECT c.relname, t.relname AS toast_table
FROM   pg_class c
JOIN   pg_class t ON t.oid = c.reltoastrelid
WHERE  c.relname = 'toast_demo';

-- 4. Count chunks in TOAST table (after inserting large values)
SELECT chunk_id, count(*) AS chunks,
       sum(length(chunk_data)) AS total_toast_bytes
FROM   pg_toast.pg_toast_XXXXX -- replace with actual OID
GROUP BY chunk_id;

-- 5. Check TOAST table size vs main table size
SELECT
    c.relname                                AS table_name,
    pg_size_pretty(pg_relation_size(c.oid))   AS main_size,
    pg_size_pretty(pg_relation_size(c.reltoastrelid)) AS toast_size,
    pg_size_pretty(pg_total_relation_size(c.oid)) AS total_size
FROM   pg_class c
WHERE  c.relname = 'toast_demo';

-- 6. Check storage strategies for all columns
SELECT
    a.attname,
    a.atttypid::regtype      AS type,
    CASE a.attstorage
        WHEN 'p' THEN 'PLAIN'
        WHEN 'e' THEN 'EXTERNAL'
        WHEN 'm' THEN 'MAIN'
        WHEN 'x' THEN 'EXTENDED'
    END                      AS toast_strategy,
    a.attcompression        AS compression
FROM   pg_attribute a
WHERE  a.attrelid = 'toast_demo'::regclass
    AND a.attnum > 0
    AND NOT a.attisdropped
ORDER BY a.attnum;

-- 7. Demonstrate EXTERNAL (no compression)
ALTER TABLE toast_demo ALTER COLUMN binary SET STORAGE EXTERNAL;
-- Re-insert the large binary
UPDATE toast_demo SET binary = repeat('\xDEADBEEF', 2000)::bytea WHERE id = 2;

-- 8. TOAST access timing (shows de-toast overhead)
EXPLAIN (ANALYZE, BUFFERS)
SELECT id, length(large) FROM toast_demo WHERE id = 2;
-- Look at "Heap Fetches" and buffer hits for the TOAST table

```

## When TOAST Hurts Performance

### 1. Repeatedly fetching large columns you don't need

```
-- BAD: fetches entire TOAST value, decompresses it, discards it
SELECT id, created_at FROM events;

-- GOOD: exclude the large column
SELECT id, created_at FROM events;
-- (same if events has a toastable 'payload' column – avoid SELECT *)
```

### 2. Filtering on large TOAST columns

```
-- BAD: de-toasts every row, then applies the filter
SELECT id FROM documents WHERE body LIKE '%keyword%';

-- BETTER: Full-text search index
CREATE INDEX ON documents USING GIN (to_tsvector('english', body));
SELECT id FROM documents WHERE to_tsvector('english', body) @@
to_tsquery('keyword');
```

### 3. Updating a single field in a wide row with TOAST

```
-- If 'payload' is large and toasted, this UPDATE re-writes and re-toasts 'payload'
-- even though only 'status' changed
UPDATE jobs SET status = 'done' WHERE id = 1;
-- PostgreSQL must write a new tuple version; if payload is EXTENDED it gets copied
too
```

### 4. Aggregations over TOAST columns

```
-- Forces de-toast of every row's json_data
SELECT count(*), avg(length(json_data)) FROM audit_log;
```

### 5. EXTERNAL strategy on frequently updated data

EXTERNAL skips compression, so the stored value is larger → more chunks → more TOAST I/O per de-toast.

---

## Common Mistakes

1. **SELECT \*** on tables with large columns. This de-toasts every toasted column for every row, even if the calling code ignores most of the data.
2. Using **EXTERNAL** on compressible text. Unless the text is truly incompressible, EXTENDED is almost always better — it reduces both storage and I/O.



3. **Assuming TOAST is free.** It trades write overhead (chunking, compression, TOAST table insert) for the ability to store large values. At high write rates the TOAST table can be a significant bottleneck.
4. **Forgetting VACUUM must cover TOAST tables.** TOAST tables accumulate dead tuples from row updates just like main tables. Autovacuum does run on TOAST tables, but if it is misconfigured or disabled, TOAST tables bloat.
5. **Not accounting for TOAST size in `pg_relation_size()`** . `pg_relation_size('mytable')` returns only the main fork size. Use `pg_total_relation_size('mytable')` to include TOAST and indexes.
6. **Changing storage strategy does not rewrite existing rows.** `ALTER TABLE t ALTER COLUMN c SET STORAGE EXTERNAL` only affects new or updated tuples. Existing rows keep their old storage form.

---

## Best Practices

- **Use `lz4` compression** for frequently accessed large values (PostgreSQL 14+): `ALTER TABLE t ALTER COLUMN c SET COMPRESSION lz4;`
- **Use `EXTERNAL`** only for already-compressed binary data (images, ZIP files, encrypted blobs) where compression would be wasteful.
- **Avoid `SELECT *`** on tables known to have large toasted columns.
- **Create functional indexes or materialized attributes** for frequently-queried subsets of large JSON/JSONB documents rather than de-toasting the whole value.
- **Monitor TOAST table size** separately: `pg_total_relation_size` vs `pg_relation_size` .
- **Set `fillfactor` lower** on TOAST-heavy tables: more dead tuple churn means more TOAST table I/O even if HOT updates are not applicable.

---

## Performance Considerations

| Scenario                                           | Impact                                                   | Mitigation                                                        |
|----------------------------------------------------|----------------------------------------------------------|-------------------------------------------------------------------|
| Large JSONB documents frequently read              | Decompression overhead per row                           | lz4 compression, cache results in app layer                       |
| Many small updates to rows with large toasted cols | Each update copies toast pointer, may create dead chunks | Consider splitting large cols to a separate table                 |
| Full table scan filtering on toasted column        | De-toast every row                                       | GIN/GiST index, or store derived searchable attributes separately |
| TOAST table fragmentation from many deletes        | Wasted I/O                                               | Run <code>VACUUM</code> on main table (cascades to TOAST)         |
| Replication lag from large TOAST writes            | Large WAL volume for TOAST inserts                       | Consider <code>STORAGE EXTERNAL</code> for truly write-once blobs |
| <code>LIKE</code> on toasted text columns          | O(n) de-toast cost                                       | <code>pg_trgm</code> GIN index                                    |

---

## Interview Questions

**Q1.** What is the TOAST threshold?

Approximately 2 KB ( `BLCKSZ / 4` ). When a tuple's total size would exceed this, PostgreSQL begins applying TOAST strategies to variable-length columns to reduce it.

**Q2.** What are the four TOAST strategies and when would you use EXTERNAL?

PLAIN (no compression or out-of-line), EXTENDED (compress then out-of-line, default), EXTERNAL (out-of-line without compression), MAIN (compress first, out-of-line last resort). Use EXTERNAL for data that is already compressed (JPEG, ZIP, encrypted binary) where compression would inflate the size.

**Q3.** Describe the schema of a TOAST table.

Three columns: `chunk_id oid` (identifies which logical value this chunk belongs to), `chunk_seq int4` (0-based order of chunks), `chunk_data bytea` (up to 2000 bytes of data per chunk). The table has a unique index on ( `chunk_id`, `chunk_seq` ) .

**Q4.** Is TOAST access visible in SQL queries?

No, it is completely transparent. The executor automatically de-toasts values when returning them to the client. There is no special syntax. The overhead is visible in `EXPLAIN (ANALYZE, BUFFERS)` as buffer hits against the TOAST table.

**Q5.** How does PostgreSQL represent a toasted value in the main heap?

As an 18-byte "toast pointer" struct ( `varattrib_4b` with the external flag set). It contains the `chunk_id` (to look up chunks in the TOAST table), the original size, and the compressed size.

**Q6.** Why does `pg_relation_size('mytable')` not include TOAST data?

`pg_relation_size` returns only the main fork of the named relation. The TOAST table is a separate relation with its own OID. Use `pg_total_relation_size` which sums the main table, all forks, all indexes, and the TOAST table.

**Q7.** What happens to TOAST data when you UPDATE a row?

A new tuple version is written in the main heap. If the toasted column value did not change, the new version still contains a toast pointer referencing the same `chunk_id` in the TOAST table — the TOAST chunks are shared. Only if the column value changes are new chunks written and the old chunks marked for deletion (vacuumed later).

**Q8.** How does `lz4` differ from `pglz` for TOAST?

Both are lossless compression algorithms. `lz4` (available from PG 14) is significantly faster — 3–5× faster decompression — at a small cost in compression ratio. `pglz` produces slightly smaller output on average. For read-heavy workloads or large JSON documents, `lz4` is almost always the better choice.

**Q9.** Can you query a TOAST table directly?

Superusers can, using the schema-qualified name `pg_toast.pg_toast_<oid>` . Regular users cannot access the `pg_toast` schema. In practice, directly querying TOAST tables is used only for forensics/debugging — normally you let the executor handle it transparently.

**Q10.** What performance issue can `SELECT *` cause on a table with TOAST columns?

It forces decompression and reassembly of every toasted value in every row. If you only need non-toasted columns, this wastes I/O bandwidth reading TOAST table pages and CPU time decompressing data.

---

## Exercises with Solutions

### Exercise 1 — Observe TOAST in action

**Task:** Create a table, insert a row with a 50 KB text value, and confirm it is stored in the TOAST table.

```
-- Solution
CREATE TABLE big_doc (id serial, content text);
INSERT INTO big_doc (content) VALUES (repeat('PostgreSQL TOAST demo. ', 2500));

-- Find the TOAST table OID
SELECT reltoastrelid FROM pg_class WHERE relname = 'big_doc';
-- e.g., returns 16390

-- Count chunks
SELECT count(*) AS chunk_count,
       sum(length(chunk_data)) AS compressed_bytes
FROM pg_toast.pg_toast_16390; -- use actual OID

-- Compare logical vs stored size
SELECT id, length(content) AS logical_bytes,
       pg_column_size(content) AS stored_bytes
FROM big_doc;
```

---

### Exercise 2 — EXTERNAL vs EXTENDED comparison

**Task:** Compare storage and query speed for the same data using EXTERNAL and EXTENDED strategies.

```
-- Create two tables
CREATE TABLE toast_extended (id int, data text);      -- default EXTENDED
CREATE TABLE toast_external (id int, data text);
ALTER TABLE toast_external ALTER COLUMN data SET STORAGE EXTERNAL;

-- Insert 1000 rows with 5 KB compressible text
INSERT INTO toast_extended
SELECT i, repeat('test data ', 500) FROM generate_series(1,1000) i;
INSERT INTO toast_external
SELECT i, repeat('test data ', 500) FROM generate_series(1,1000) i;

-- Compare total sizes
SELECT 'extended' AS strategy,
       pg_size_pretty(pg_total_relation_size('toast_extended')) AS size
UNION ALL
SELECT 'external',
       pg_size_pretty(pg_total_relation_size('toast_external'));

-- Compare read speed
\timing
SELECT sum(length(data)) FROM toast_extended;
```

```
SELECT sum(length(data)) FROM toast_external;
-- EXTERNAL will be faster to read (no decompression) but use more disk space
```

---

### Exercise 3 — TOAST table maintenance

**Task:** Insert 1000 large rows, delete 500, and observe TOAST table bloat before and after VACUUM.

```
CREATE TABLE bloat_test (id int, payload text);
INSERT INTO bloat_test
SELECT i, repeat('data', 1000) FROM generate_series(1,1000) i;

-- Size before
SELECT pg_size_pretty(pg_total_relation_size('bloat_test')) AS before_size;

-- Delete half the rows
DELETE FROM bloat_test WHERE id <= 500;

-- Size after delete (dead tuples in TOAST not yet reclaimed)
SELECT pg_size_pretty(pg_total_relation_size('bloat_test')) AS after_delete;

-- Vacuum
VACUUM VERBOSE bloat_test;

-- Size after vacuum
SELECT pg_size_pretty(pg_total_relation_size('bloat_test')) AS after_vacuum;
-- VACUUM automatically processes the associated TOAST table
```

---

## Cross-References

- **02\_heap\_storage.md** — Varlena headers within the main tuple
- **06\_vacuum.md** — VACUUM processes TOAST tables alongside main tables
- **09\_free\_space\_map.md** — TOAST tables have their own FSM
- **10\_system\_catalogs.md** — `pg_class.reltoastrelid`, `pg_attribute.attstorage`